Task 1: News Topic Classifier Using BERT

Objective

Fine-tune `bert-base-uncased` to classify news headlines into one of 4 topics:
World, Sports, Business, Sci/Tech.

Dataset

AG News (amananandrai/ag-news-classification-dataset)
Train: ~120,000 samples
Test: ~7,600 samples
Classes: 4 (balanced)

Dataset Loading & Preprocessing

- Downloaded via `kagglehub`
- Combined `Title` + `Description` → `text` column
- Labels: Class Index (1–4) → 0–3
- Tokenized with `bert-base-uncased` tokenizer (max_length=128, truncation & padding)

[ ]

```
import kagglehub

path = kagglehub.dataset_download("amananandrai/ag-news-classification-dataset")
print("Dataset downloaded to:", path)
```

Commands + Code + Text Run all Connect

```
path = kagglehub.dataset_download("amananandrai/ag-news-classification-dataset")
print("Dataset downloaded to:", path)
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.0).
Downloading from https://www.kaggle.com/api/v1/datasets/download/amananandrai/ag-news-classification-dataset?dataset_version_number=2...
100%|██████████| 11.4M/11.4M [00:01<00:00, 7.96MB/s]Extracting files...

Dataset downloaded to: /root/.cache/kagglehub/datasets/amananandrai/ag-news-classification-dataset/versions/2
```

```
!pip install transformers datasets evaluate accelerate -q
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 3.5 MB/s eta 0:00:00
```

```
import pandas as pd
import os
print("Files in dataset folder:")
print(os.listdir(path))
train_file = os.path.join(path, "train.csv")
test_file = os.path.join(path, "test.csv")
```

```
Files in dataset folder:
['test.csv', 'train.csv']
```

```
train_df = pd.read_csv(train_file)
test_df = pd.read_csv(test_file)

train_df['text'] = train_df['Title'].fillna('') + " " + train_df['Description'].fillna('')
test_df['text'] = test_df['Title'].fillna('') + " " + test_df['Description'].fillna('')
```

```
train_df['label'] = train_df['Class Index'] - 1
test_df['label'] = test_df['Class Index'] - 1
```

Commands + Code + Text Run all Connect

```
train_df['label'] = train_df['Class Index'] - 1
test_df['label'] = test_df['Class Index'] - 1

print("\nTrain shape:", train_df.shape)
print("Test shape:", test_df.shape)
print("\nSample rows:")
display(train_df.head(3))

print("\nLabel distribution:")
print(train_df['label'].value_counts())
```

```
Train shape: (120000, 5)
Test shape: (7600, 5)

Sample rows:
```

	Class Index	Title	Description	text	label
0	3	Wall St. Bears Claw Back Into the Black (Reuters)	Reuters - Short-sellers, Wall Street's dwindli...	Wall St. Bears Claw Back Into the Black (Reute...	2
1	3	Carlyle Looks Toward Commercial Aerospace (Reu...	Reuters - Private investment firm Carlyle Grou...	Carlyle Looks Toward Commercial Aerospace (Reu...	2
2	3	Oil and Economy Cloud Stocks' Outlook (Reuters)	Reuters - Soaring crude prices plus worries\ab...	Oil and Economy Cloud Stocks' Outlook (Reuters...	2

```
Label distribution:
label
2    30000
3    30000
1    30000
0    30000
Name: count, dtype: int64
```

colab.research.google.com/drive/1Kmt9SSJmsZCrxRAUbb73DFwL1R4QQ4Or

Commands + Code + Text Run all Connect

```
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart you
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% ||||||
```

colab.research.google.com/drive/1Kmt9SSJmsZCrxRAUbb73DFwL1R4QQ4Or

Commands + Code + Text Run all Connect

```
from datasets import Dataset

train_dataset = Dataset.from_pandas(train_df)
test_dataset = Dataset.from_pandas(test_df)

def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )
```

```
tokenized_train = train_dataset.map(tokenize_function, batched=True)
tokenized_test = test_dataset.map(tokenize_function, batched=True)

tokenized_train.set_format("torch", columns=["input_ids", "attention_mask", "label"])
tokenized_test.set_format("torch", columns=["input_ids", "attention_mask", "label"])

print("Re-tokenized with DistilBERT")
```

```
Map: 100% 120000/120000 [01:04<00:00, 3969.85 examples/s]
Map: 100% 7600/7600 [00:01<00:00, 3978.60 examples/s]
Re-tokenized with DistilBERT
```

```
from transformers import TrainingArguments

subset_size = 8000
tokenized_train_small = tokenized_train.shuffle(seed=42).select(range(subset_size))
```

Variables Terminal

colab.research.google.com/drive/1Kmt9SSJmsZCrxRAUbb73DFwL1R4QQ4Or

Commands + Code + Text Run all Connect

```
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size = 64,
    num_train_epochs = 1,
)

training_args.gradient_accumulation_steps = 2
```

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train_small,
    eval_dataset=tokenized_test,
)

trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned
  super().__init__(loader)
```

[63/63 1:27:17, Epoch 1/1]

Step	Training Loss

Writing model shards: 100% 1/1 [00:06<00:00, 6.46s/it]

```
TrainOutput(global_step=63, training_loss=1.10600099109468, metrics={'train_runtime': 5328.7626, 'train_samples_per_second': 1.501, 'train_steps_per_second': 0.012,
'total_flos': 264944246784000.0, 'train_loss': 1.10600099109468, 'epoch': 1.0})
```

colab.research.google.com/drive/1Kmt9SSJmsZCrxRAUbb73DFwL1R4QQ4Or

Commands + Code + Text Run all Connect

Evaluation Metrics

- Accuracy
- Weighted F1-score (not computed in this run, but can be added)

Final Test Results

(From trainer.evaluate())

- accuracy: [0.8992]
- eval_loss: [0.3236]
- eval_runtime: [1604.3338

]

[ ]

```
eval_results = trainer.evaluate()

print("\nFinal Evaluation Results on Test Set:")
for key, value in eval_results.items():
    if isinstance(value, float):
        print(f"{key}: {value:.4f}")
    else:
        print(f"{key}: {value}")
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned
  super().__init__(loader)
```

[950/950 26:01]

```
Final Evaluation Results on Test Set:
eval_loss: 0.3301
eval_runtime: 1563.0556
eval_samples_per_second: 4.8620
```

Variables Terminal

colab.research.google.com/drive/1Kmt9SSJmsZCrxRAUbb73DFwL1R4QQ4Or

Commands + Code + Text Run all Connect

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned
  super().__init__(loader)
```

[950/950 26:01]

```
Final Evaluation Results on Test Set:
eval_loss: 0.3301
eval_runtime: 1563.0556
eval_samples_per_second: 4.8620
eval_steps_per_second: 0.6080
epoch: 1.0000
```

[ ]

```
trainer.save_model("./fine_tuned_distilbert_agnews")
tokenizer.save_pretrained("./fine_tuned_distilbert_agnews")

print("Model and tokenizer saved to ./fine_tuned_distilbert_agnews")
```

Writing model shards: 100% 1/1 [00:05<00:00, 5.60s/it]

```
Model and tokenizer saved to ./fine_tuned_distilbert_agnews
```

Final Summary & Insights

- DistilBERT achieved solid accuracy with minimal training (1 epoch, 8k subset)
- Fast and lightweight model compared to full BERT
- Combining title + description helps capture context
- Limitations: short training & small subset — full run would give higher scores
- Skills gained: Transformers fine-tuning, Trainer API, Gradio deployment

Variables Terminal

colab.research.google.com/drive/1Kmt9SSJmsZCrxRAUbb73DFwL1R4QQ4Or

Commands + Code + Text Run all Connect

Live Demo – Gradio Interface

Interactive tool to classify any news text in real time.

Enter headline/description → get predicted topic + top 3 confidences.

[ ]
```
!pip install gradio -q
```

[ ]
```
import gradio as gr
from transformers import pipeline
import torch
```

[ ]
```
classifier = pipeline(
    "text-classification",
    model="./fine_tuned_distilbert_agnews",
    tokenizer="./fine_tuned_distilbert_agnews",
    device=0 if torch.cuda.is_available() else -1,
    return_all_scores=True
)

id2label = {
    0: "World",
    1: "Sports",
    2: "Business",
    3: "Sci/Tech"
}

def predict(text):
```

Variables Terminal

colab.research.google.com/drive/1Kmt9SSJmsZCrxRAUbb73DFwL1R4QQ4Or

Commands + Code + Text Run all Connect

```python
def predict(text):
    if not text.strip():
        return "Please enter some news text!"
    results = classifier(text)[0]
    sorted_results = sorted(results, key=lambda x: x['score'], reverse=True)

    top1 = sorted_results[0]
    top3 = sorted_results[:3]

    output = f"**Predicted Topic:** {id2label[int(top1['label'].split('_')[-1])]} (Confidence: {top1['score']:.4f})\n\n"
    output += "**Top 3 Predictions:**\n"
    for i, res in enumerate(top3, 1):
        label_idx = int(res['label'].split('_')[-1])
        output += f"{i}. {id2label[label_idx]} ({res['score']:.4f})\n"

    return output
```

Loading weights: 100% 104/104 [00:00<00:00, 154.05it/s, Materializing param=pre_classifier.weight]

```python
import gradio as gr

demo = gr.Interface(
    fn=predict,
    inputs=gr.Textbox(
        label="Enter news headline or description",
        placeholder="Example: Apple launches new AI-powered MacBook...",
        lines=5
    ),
    outputs=gr.Textbox(label="Prediction Results"),
    title="News Topic Classifier (DistilBERT on AG News)",
    description="Type any news text to see the predicted category.",
```

Variables Terminal

Commands + Code + Text Run all Connect

Model Development & Training

- Model: `distilbert-base-uncased` + classification head (4 labels)
- Framework: Hugging Face Trainer API
- Training settings:
  - Subset size: 8,000 samples
  - Epochs: 1
  - Batch size: 64 (with gradient accumulation)
  - Trained on Colab GPU
- Model saved to `./fine_tuned_distilbert_agnews`

[ ]

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=4
)

print("DistilBERT tokenizer and model loaded (smaller & faster)")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart you
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

Variables Terminal

News Topic Classifier (DistilBERT on AG News)

Type any news text to see the predicted category.

Enter news headline or description

Example: Apple launches new AI-powered MacBook...

Prediction Results

Flag

Clear **Submit**

Examples

- Pakistan cricket team wins T20 World Cup final
- Stock market crashes amid global recession fears
- Iran has dropped missiles on Isreal beack to back
- Tesla announces breakthrough in solid-state battery technolo...